

SKIjack:

A low-level native language for the SKI calculus

N. E. DAVIS

ZeroAttest, United States of America
(email: neal.davis@protonmail.com)

Abstract

We present SKIjack, a language for writing interpreted code over the *SKI* calculus to be evaluated in a metacircular fashion, whose feature set is derived from an existing artifact rather than designed: the 618-atom self-interpreter of Davis (2027), decomposed into its fuel loop, its dispatch, its spine walker and its atoms, with each part promoted to a surface construct. Over a homoiconic calculus the designer chooses where quotation happens. Over a calculus without self-quotation this choice is unavailable: no combinator can take another apart, so reification is metalevel by theorem and a language’s staging boundary is fixed. Declaring an object language generates that furniture, so authoring the semantics of a new symbol—the core operation performed by hand in the previous paper—becomes merely a two-line edit. Quotation is available at compile time only, and at the positions where a function would be asked to behave as data the checker statically refuses every occupant it can prove to be one. A bound namespace is a language feature: a path is written as a literal of a declared path type and compared by structural equality on its encoding, and a blocked run is replayed under a namespace that only ever grows.

1 Introduction

Over a homoiconic calculus (Lisp, Nock) the metacircular tradition (McCarthy 1960; Yarvin 2018) gets quotation for free and the language designer chooses where evaluation stages live. Over a non-homoiconic calculus no such choice is possible: Proposition 3.1 of Davis (2027), which follows from Theorem 3.2 of Jay and Given-Wilson (2011), demonstrates that no closed term quotes *every* live term, so the compile-time/runtime line is fixed prior to the language’s design; the two operations the calculus cannot do for itself, quote and recognize, must go to the compiler and the virtual machine respectively. Davis (2027) presented a metacircular evaluator for *SKI* which is written and evaluated in the same combinator system. This capacity immediately suggests the construction of a battery of computational conveniences and abstractions within the same system; to wit, a language. We present SKIjack (pronounced “skyjack”), a closed-supercombinator macro-like language built in and targeting the non-homoiconic *SKI* calculus as its instruction set architecture. Every named entity compiles to a closed term and every name is resolved before anything runs, so the language has no runtime environment to address: bracket abstraction has already closed what a subject-oriented calculus would have to carry. SKIjack is evaluable on any *SKI* interpreter as a virtual machine.

The `whnfF` metacircular interpreter and `UQ` unquoter function presented in Davis (2027) are the starting point for our work. That paper discussed extensions to the *SKI* calculus built on higher layers of interpreters, introducing an `Err` error type, a `Maybe` partiality type, and



© 2026 Copyright held by the owner/author(s).

This work is licensed under a Creative Commons Attribution 4.0 International License.

discussed implications such as namespace value retrieval and other innovations. By authoring a new symbol at one site, the `whnf` approach enables extension and modification of the combinator’s semantics, permitting it to behave more like a language than a mere combinator system. This paper presents such a language, SKIjack, and three contributions for SKI combinator research. First, object-language declarations that generate an interpreter’s furniture—constructors, case, spine walker, rebuilder, default step equations and fuel loop—so that authoring a semantics at one site is a declaration rather than a term construction, and costs nothing over the hand construction: six programs compile to the artifacts of Davis (2027) atom for atom. Second, a two-level execution model in which a quotation is run under a named interpreter with an explicit fuel budget, so that a program’s steps, divergence and errors are observable because the interpreter is an ordinary term the program names. Third, a namespace as a language feature: a resolver is a core parameter, paths are restricted to literals of a declared path type and compared structurally, and the namespace literal compiles to a resolver.

2 Background

The calculus is the usual one. Terms are built from S , K and I by application alone, written left-associatively, with the rules $I x \rightarrow x$, $K x y \rightarrow x$ and $S f g x \rightarrow f x (g x)$. There are no variables and so no binding, no substitution and no environment; a surface language with binders reaches this calculus by bracket abstraction, which is the whole of the compiler’s back end (Turner 1979a).

Data are Scott-encoded. A datum of an n -constructor type is a function of n continuations that applies the one belonging to its own constructor to its own fields, so a value *is* its own case analysis and there is no separate eliminator (Böhm and Berarducci 1985; Jansen 2013; Scott 1962). The order of the continuations is the order of the declaration, and it is part of the interface rather than a convention: a wrong continuation order is not a type error but silently different behaviour, which is why §9 treats case completeness and order as the first thing a checker owes an author.

The artifact this paper compiles to was built by Davis (2027): a fueled normal-order self-interpreter for SKI at 618 atoms, with an unquoter UQ at 43 atoms, and structural equality on encoded terms at 240. A fueled run that performs s contractions requires $s + 1$ steps. That paper reported the interpreter running a one-step object reduction in 340 host contractions, running the unquoter’s output in 91,556, and running *itself* in 504,930; it also proved the proposition around which SKIjack is organized, that no closed term quotes every live term.

We distinguish three different measures for the size of the interpreter: atoms, nodes, and distinct nodes. The interpreter is 618 *atoms*; as a tree that is 1,235 *nodes*; and sharing brings the graph to 348 *distinct nodes*, counted by identity as the term is built (by structure it is 147). Atoms are the default unit.

3 Lineage

SKIjack is a language entirely based on the SKI combinator calculus, and in particular on improving the expressiveness of SKI reasoning by the introduction of a standard machinery in the form of generated supercombinators. We briefly survey the historical and technical lineage of pure combinator computation that informs SKIjack’s design. SKIjack is far from

the first language or machine concept to lean into pure combinator computing. The present paper leans on five classic results from Corrado Böhm’s work:

- (1) *CUCH*. Böhm and Gross (1966) introduced the CUCH (C_Urry-CHurch), a surface for writing programs whose meaning is given by combinators. The *SKI* calculus (including SKIjack) operates under a more severe constraint: while CUCH’s designers were able to place its affordances such as quotation wherever they pleased, pure *SKI* calculus cannot quote expressions (Proposition 3.1 of Davis (2027)).
- (2) *Separability*. Böhm (1968) proved that if two closed λ -terms are $\beta\eta$ -normal and $\beta\eta$ -distinct then for any two terms P and Q chosen in advance there are arguments sending the first to P and the second to Q . What we take from this is directional and nothing more: structure is available from outside a term and not from within it. It does not transfer to our setting term for term, and the counterexample is the one Davis (2027) uses to motivate Proposition 3.1— $S K K$ and I are distinct normal forms of weak combinatory logic that agree on every argument, so no applicative context separates them. What licenses the practice of this paper is therefore not Böhm’s theorem but the metalevel: a host may read a term’s syntax. Our fresh-marker probes are a behavioural check and not a decision procedure for term identity, which we settle by structural hash.
- (3) *Term-algebra schema*. A type declaration generates its constructors and its case form by the schema Böhm and Berarducci (1985) gave for term algebras, in the Scott variant (Jansen 2013; Scott 1962) rather than the Church one: an interpreter’s step function wants case analysis in place of a fold which fixes the declaration order as the continuation order.
- (4) *First self-compiling compiler*. The first compiler written in its own language is recognized as Böhm (1954), his 1951 dissertation at ETH Zürich (Knuth and Pardo 1980). In the case of SKIjack, the compiler carries out quotation at compile time.
- (5) *Self-interpreter with a normal form*. Finally, Berarducci and Böhm (1993) gave a self-interpreter of the λ -calculus having a normal form.

Bracket abstraction permits the procedural compilation of a low-level applicative language to combinators (Turner 1979b); there is a related abstraction algorithm (Turner 1979a). That technique was realized in hardware as the “*SKI* Machine” (SKIM), a graph-reduction machine built on a fixed “Turner set” of combinators (Clarke et al. 1980), and in its successor (Stoye 1985; Stoye et al. 1984). Hughes (1982) then introduced the supercombinator—the closed abstraction compiled as one unit—expressly as the alternative to a fixed set, whose expansions grow with the program; the compiled-graph-reduction line that followed runs through the G-machine (Johnsson 1984). Our own position is the awkward one of taking both: we lift as Hughes does and then abstract each supercombinator into $\{S, K, I\}$, re-incurring the growth he set out to avoid, and pay down this debt with the virtual machine semantics of §11. (A standard account of the whole pipeline has been given by Peyton Jones (1987).) The SKIM tradition operated with a fixed set of recognized combinators executed natively in microcode, an accelerated process comparable to the “jet” approach we will adopt in this work. Much of this work was concerned with the cost of any particular abstraction, in combinators per source symbol (Joy et al. 1985; Noshita 1985), and we report atom counts in line with that tradition.

In the preceding research program, the interpreter lived either in a machine (Turner) or in microcode (SKIM), with the reduction engine as an external given. The object introduced by Davis (2027) is a 618-atom self-interpreter which exists as a term within the language it interprets. Previous supercombinator work has had no reason to write such an artifact; its self-application question was the compiler bootstrapping in its own source language, not the reducer reducing itself, and so the tower, the per-level cost, and the encoding boundary never arose there.

However, pursuant to the introduction of a self-interpreter as a term within the language, new considerations arise. Staging becomes a design problem: Turner’s SASL could not quote either, but because no SASL program was trying to run another SASL program under a fuel budget, there was no obligation to address the lack. Declaring an object language becomes a language feature, since an interpreter’s constructors, case form, spine walker, rebuilder and fuel loop can be generated from the declaration rather than written. Finally, the native compiler owes its output a strict discipline: expansion is bijective, terms lift back to named source, and the dictionary is published and keyed by structural hash. This discipline becomes crucial because the emitted term is simultaneously the object of study and the key a VM matches on, whereas nobody expected to read the output of a SASL compiler.

This last merits an aside. A virtual machine (§11) may identify supercombinators of arbitrary size and reduce them against a table of known outcomes in native code (what we call a “jet”, short for “jet-accelerated code”, a term from the Nock computing lineage). Mechanically, such a procedure corresponds to Turner’s extended combinator set and SKIM’s microcode. However, our theorem does not licence arbitrary native code replacement; the interpreter jet performs a structural recognition that no term can perform for itself and thus is not an optimization of an operation the language has. The operational consequence of this prohibits a jet from quoting on behalf of a running term. The VM is addressed briefly in §11.

The metacircular and reflective-tower literature is surveyed in Davis (2027) and not repeated here; this section covers the combinator-machine lineage specific to SKIjack’s design.

4 The interpreter

The `whnfF` function (Listing 2) forms the centerpiece of the interpreter, responsible for reducing terms to weak head normal form. It orchestrates the evaluation strategy for the language, ensuring that terms are reduced correctly according to the operational semantics. It also suggests the language features that are necessary to support such an evaluation strategy, which will emerge in SKIjack as primitives. There is also an unquoter function `UQ`, responsible for converting weak head normal forms back into their original syntactic representation. This is necessary for maintaining the correspondence between the evaluated terms and their original syntactic forms, ensuring that the interpreter’s output can be related back to the source code. In Davis (2027), we simply named the features by their operational significance as a shorthand; here they become language elements in their own right.

Table 1 gives some typed forms with their defining equations and with closed $\{S, K, I\}$ terms emitted by the expander (i.e., compiled output). Three of the six reduce to the same term, and two more are a second same term. This results because a Scott datum is its own case analysis, so the first constructor of any two-constructor type discards the remaining continuation and is K , whatever the type was called, and a one-field second constructor applies one continuation to one field and is the same eight atoms whether the field is a numeral or

a term. SKIjack maintains the type discipline at compile time (§9), but at the level of compiled terms the type distinctions are erased. A checker may reject a program that offers a numeral where a list is wanted, but nothing downstream of the expander could tell the two apart, because downstream of the expander they are one object.

Table 1. Some significant forms, their definitions, and the terms they compile to. The first constructor of each pair is the same single atom and the one-field second constructors are the same eight; only `cons`, which carries two fields, is distinct.

form	definition	expansion	atoms
<code>nil</code>	$\lambda n.\lambda c.n$	<code>K</code>	1
<code>cons h t</code>	$\lambda n.\lambda c.c\ h\ t$	<code>S (K (S (K K))) (S (S (K S) (S (K K) (S (K S) (S (K (S I)) K))) (K K)))</code>	22
<code>nothing</code>	$\lambda n.\lambda j.n$	<code>K</code>	1
<code>just x</code>	$\lambda n.\lambda j.j\ x$	<code>S (K K) (S (K (S I)) K)</code>	8
<code>zero</code>	$\lambda z.\lambda s.z$	<code>K</code>	1
<code>suc n</code>	$\lambda z.\lambda s.s\ n$	<code>S (K K) (S (K (S I)) K)</code>	8

The metacircular evaluator uses these object terms to represent the basic constructs of the language within itself. Next, `whnfF` itself is composed of 618 atoms built from a few pieces. Table 2 inventories `whnfF`'s parts, smallest first, with the term each part compiles to. By reading the lower block from the bottom upwards, the interpreter assembles itself out of the upper one.

- | | |
|--|---|
| (1) <code>hd</code> , the head extractor; | (14) <code>stepK2</code> , the second <i>K</i> step equation; |
| (2) <code>tl</code> , the tail extractor; | (15) <code>stepI</code> , the <i>I</i> step equation; |
| (3) <code>resI</code> , the <i>I</i> result equation; | (16) <code>stepK1</code> , the first <i>K</i> step equation; |
| (4) <code>resK</code> , the <i>K</i> result equation; | (17) <code>stepK</code> , the <i>K</i> step equation; |
| (5) <code>Y</code> , the fixed-point combinator; | (18) <code>sp</code> , the spine walker; |
| (6) <code>resS</code> , the <i>S</i> result equation; | (19) <code>stepS3</code> , the third <i>S</i> step equation; |
| (7) <code>pair</code> , the pair constructor; | (20) <code>stepS2</code> , the second <i>S</i> step equation; |
| (8) <code>App</code> , the application constructor; | (21) <code>stepS1</code> , the first <i>S</i> step equation; |
| (9) <code>UQ</code> , the unquoter; | (22) <code>stepS</code> , the <i>S</i> step equation; |
| (10) <code>rb1</code> , the first rebuilder equation; | (23) <code>step</code> , the step walker from the spine to the head leaf; |
| (11) <code>spApp</code> , the spine application handler; | (24) <code>loop1</code> , the first loop equation; and |
| (12) <code>rb</code> , the rebuilder; | (25) <code>whnfF</code> , the whole interpreter. |
| (13) <code>stepI1</code> , the first <i>I</i> step equation; | |

Note that the expansions of `rb`, `UQ`, `sp`, and `whnfF` begin with the fourteen atoms of *Y*. Appendix B of Davis (2027) observes this of `UQ` and `whnfF`; the inventory here shows it holds of `rb` and `sp` as well, and of nothing else in the table, so it is a property of every recursive equation rather than of two particular terms. The fixpoint tie is a prefix of the emitted term, and which supercombinators recurse can be read off the object itself. This is perhaps the one piece of the surface's structure that survives bracket abstraction in a form a reader can observe directly. It does not run the other way: `lift` names exact structural matches, so a lifted

Table 2. Every named part of the `whnfF` interpreter, ordered by size. Above the rule, the closed $\{S, K, I\}$ term in full; below it, the same term named against the parts above by *lift*, which renames exact structural matches and nothing else. Where several names share one term—`Just` and `suc` are the same eight atoms—the source’s own name is shown.

part	atoms	expansion
hd	4	$S\ I\ (K\ K)$
t1	5	$S\ I\ (K\ (K\ I))$
resI	11	$S\ (K\ K)\ (S\ (K\ K)\ (S\ (K\ (S\ I))\ K))$
resK	13	$S\ (K\ K)\ (S\ (K\ (S\ (K\ K)))\ (S\ (K\ (S\ I))\ K))$
Y	14	$S\ (K\ (S\ I\ I))\ (S\ (S\ (K\ S)\ K)\ (K\ (S\ I\ I)))$
resS	15	$S\ (K\ (S\ (K\ K)))\ (S\ (K\ (S\ (K\ K)))\ (S\ (K\ (S\ I))\ K))$
pair	17	$S\ (S\ (K\ S)\ (S\ (K\ K)\ (S\ (K\ S)\ (S\ (K\ (S\ I))\ K))))\ (K\ K)$
App	32	$S\ (K\ (S\ (K\ K)))\ (S\ (K\ (S\ (K\ K)))\ (S\ (K\ (S\ (K\ K)))\ (S\ (S\ (K\ S)\ (S\ (K\ K)\ (S\ (K\ S)\ (K\ (S\ I))\ K))))\ (K\ K))))$
UQ	43	$S\ (K\ (S\ I\ I))\ (S\ (S\ (K\ S)\ K)\ (K\ (S\ I\ I)))\ (S\ (K\ (S\ (S\ (S\ (S\ I\ (K\ S))\ (K\ K))\ (K\ I))))\ (S\ (K\ K)\ (S\ (S\ (K\ S)\ (S\ (K\ (S\ (K\ S)))\ (S\ (K\ K))))\ K))))$
rb1	44	$S\ (S\ (K\ S)\ (S\ (K\ K)\ (S\ (K\ S)\ K)))\ (K\ (S\ (K\ (S\ (K\ K)))\ (S\ (K\ (S\ (K\ K)))\ (S\ (K\ (S\ (K\ K)))\ (S\ (S\ (K\ S)\ (S\ (K\ K)\ (S\ (K\ S)\ (S\ (K\ (S\ I))\ K))))\ (K\ K))))))$
spApp	48	$S\ (S\ (K\ S)\ (S\ (K\ K)\ (S\ (K\ S)\ (S\ (K\ (S\ (K\ S)))\ (S\ (K\ K))))\ (K\ (S\ (K\ K)\ (S\ (K\ (S\ (S\ (K\ (S\ (K\ (S\ (K\ K)))\ (S\ (S\ (K\ S)\ (S\ (K\ K)\ (S\ (K\ S)\ (S\ (K\ (S\ I))\ K))))\ (K\ K))))\ K))))$
rb	74	$S\ (K\ (S\ I\ I))\ (S\ (S\ (K\ S)\ K)\ (K\ (S\ I\ I)))\ (S\ (K\ (S\ (S\ (K\ S)\ (S\ (K\ (S\ I))\ K))))\ (S\ (K\ (S\ (K\ K)))\ (S\ (S\ (K\ S)\ (S\ (K\ K)\ (S\ (K\ S)\ K)))\ (K\ (S\ (K\ (S\ (K\ K)))\ (S\ (K\ (S\ (K\ K)))\ (S\ (K\ (S\ (K\ K)))\ (S\ (S\ (K\ S)\ (S\ (K\ K)\ (S\ (K\ S)\ (S\ (K\ (S\ I))\ K))))\ (K\ K))))))$
stepI1	86	$S\ (K\ (S\ (K\ Just)))\ rb$
stepK2	89	$S\ (K\ K)\ stepI1$
stepI	92	$S\ hd\ (K\ stepI1)$
stepK1	99	$S\ (K\ (S\ hd))\ (S\ (K\ K)\ stepK2)$
stepK	105	$S\ hd\ (K\ stepK1)$
sp	126	$Y\ (S\ (K\ (S\ (S\ (K\ S)\ (S\ (S\ (K\ S)\ (S\ (S\ (K\ S)\ (S\ (S\ (K\ S)\ K)\ (K\ resS))))\ (K\ resK)))\ (K\ resI))))\ (S\ (K\ K)\ spApp)$
stepS3	207	$S\ (K\ (S\ (K\ (S\ (K\ (S\ (K\ Just))))))\ (S\ (K\ (S\ (K\ (S\ (K\ rb))))\ (S\ (S\ (K\ S)\ (S\ (K\ K)\ (S\ (K\ S)\ (S\ (K\ (S\ (K\ App)))\ App))))\ (K\ App)))$
stepS2	221	$S\ (K\ (S\ (K\ (S\ hd))))\ (S\ (K\ (S\ (K\ K)))\ stepS3)$
stepS1	231	$S\ (K\ (S\ hd))\ (S\ (K\ K)\ stepS2)$
stepS	237	$S\ hd\ (K\ stepS1)$
step	569	$S\ (S\ (S\ (S\ sp\ (K\ K))\ (K\ stepS))\ (K\ stepK))\ (K\ stepI)$
loop1	588	$S\ (K\ (S\ (S\ (K\ S)\ (S\ (K\ K)\ (S\ step\ Just))))\ K$
whnfF	618	$Y\ (S\ (K\ (S\ (S\ (K\ S)\ (S\ (K\ K)\ hd)))\ (S\ (K\ K)\ loop1))$

rendering can name a subterm for a coincidence—the `hd` in the `whnfF` row is $\lambda n. n$ `Nothing`, and reads as `hd` only because `Nothing` is `K`.

5 The language

The `whnfF` function of Davis (2027) introduced a number of independent operational terms within its full expression, and these suggest language features such as its `loop`, its comparison

and dispatch, and its atoms (`suc`, `zero`, `nil`, `just`, etc.). These features were composed into larger machine terms, which are similar to the supercombinators of Hughes (1982).

SKIjack’s features are written as a set of object terms in the metacircular evaluator; in essence, it is a macro language with a gently typed surface discipline. They name and highlight the components of `whnfF` and other recurrent supercombinators, augmented with affordances to make the language expressive and composable.

A language description of SKIjack is provided in Appendix A.

5.1 Design principles

Since quotation of live terms is not supported, the language relies on the macro-like composition of its object terms rather than runtime evaluation. A hard line between runtime quotation and compile-time quotation is forced by Proposition 3.1 of Davis (2027), which follows from Theorem 3.2 of Jay and Given-Wilson (2011): “No closed term Q over $\{S, K, I\}$ satisfies $Q M \rightarrow \langle M \rangle$ for every closed M .” Everything on the encoded side of that line is closed under the calculus, including running encoded terms, comparing them, and building cores. Everything that would turn a live value back into data is unavailable and must be staged via the virtual machine. One design problem is to make that line visible and comfortable. Another difference from traditional macro languages is that weak *SKI* reduction is call-by-name: arguments are never forced early, $K \ I \ \Omega$ is I , and no form needs thunking.

The basic elements of SKIjack are thus: a macro layer; Scott encoding; recursion through Y ; interpreters, the unquoter, and equality on encoded terms like paths; the behavioral verification discipline; and the existence of a runtime which can optionally jet the compiled code for more efficient execution.

A SKIjack program consists of a series of declarations and definitions that establish the structure and behavior of the object language terms. These programs may rely on a prelude to provide essential cores and utilities. Listing 1 shows a simple example of a namespace binding and lookup. Since the namespace literal compares paths with `EQ5`, and `wfN` must be a core in scope, a prelude is required. With those supplied, the program compiles to the results given in Table 3.

```
seg === Nat | Two | Three
path === Nil | Cons seg path
resolve := ns{/nat/two => <I>, /nat/three => <K>}
answer := wfN resolve |- <?~/nat/three>@10
```

Listing 1. A simple example of SKIjack code. This program demonstrates a simple lookup.

The quoted scry expression `<?~/nat/three>` costs 2,485 atoms because at level 1 every constructor application is itself encoded, and `resolve` costs 5,375 because the namespace literal expands to a chain that compares the incoming path against each key with `EQ5` at 240 atoms, once per key, over full encodings of both. In reduced form, it gives `RValN K` in 28,661 contractions; the sibling declaration that looks up `/nat/two` instead gives `RValN I` in 14,502, since the first key it tries is the one that matches. As can be seen, SKIjack and its `whnfF` interpreter are far from concise, but they are deterministic.

Table 3. The artifact of Listing 1. `Cons` is the same twenty-two atoms as the `cons` of Table 1: a two-field constructor in second position is one term whatever the type it belongs to was called. `answer` has been lifted against the program’s dictionary.

[illegible]

5.2 Representation ABI

A handful of representation choices sit between the compiler and everything written in the language. We call this the representation ABI because it specifies the contract between separately produced compiled artifacts, not because there are bytes or linkers. A term emitted by this compiler must interoperate with terms built by hand, and with native code that recognizes them; the two would fail to compose if either disagreed about the order of continuations. In a Scott encoding that order is simultaneously the layout of a datum and the convention for calling it.

A cell $[a \ b]$ is the Scott pair $\lambda c. c \ a \ b$, whose only operation is to hand its components to a selector; the head is $p \ K$ and the tail $p \ (K \ I)$. Axis addressing follows a binary tree numbering, with axis 1 being the whole, axis $2n$ the head of axis n and $2n + 1$ its tail, and an axis compiles to the corresponding chain of projections at one projection per bit. Axes only validly address cells; a term's application tree is code, which by Proposition 3.1 cannot be inspected by any term (Jay and Given-Wilson 2011, Thm. 3.2), and so the notation that reaches into data stops at that boundary.

A type declaration is an ordered constructor list, with the order specified in the ABI. The declaration generates each constructor as $\lambda fields. \lambda c_1 \dots c_n. c_i fields$ as well as the case form to apply a datum to n continuations in declaration order, per the schema of Böhm and Berarducci (1985) in its Scott rather than Church variant. Booleans are K for yes and K I for no, so a conditional is application to two continuations. Numerals are Scott numerals and serve as fuel; there is no native integer, and arithmetic is unary.

5.3 Kernel forms

The kernel contains about a dozen forms and is intended to remain fixed. Each form has exactly one expansion, given by a fold over the syntax tree: a name resolves in the compile-time table to a binder, a sibling equation, a constructor or a macro; juxtaposition denotes application; a cell is a pair; an axis pick is a projection chain; a lambda and an equation are bracket abstraction over their binders; a case applies a datum to one continuation per constructor in declaration order; a quotation is a Scott encoding computed at compile time;

a macro is expanded at its use site; and the two evaluation forms hand a quotation to an interpreter. Everything else is composed of these primitives.

Because every rule is a fold, the expander has no semantics of its own: the same source gives the same closed term, atom for atom, deterministically and context-free. The surface grows in macros and in the library rather than in the compiler.

5.4 *Supercombinators vs. macros*

A supercombinator and a macro can look alike in source code but are not equivalent. An equation is lambda-lifted over its binders and over the sibling equations it calls, tied with Y where it recurses, and emitted once as a supercombinator. A supercombinator is a closed term forming a single node in the graph and shared by every use. In contrast, a macro has no runtime existence but is expanded into each use site, so it costs nothing to reach and is duplicated per use. Small things (derived combinators, booleans, projections) tend to be macros, while large things (an interpreter, equality on encoded data, the standard library) prefer to be supercombinators. (Hughes proposed supercombinators as the alternative to compiling into a fixed combinator set, whose expansions blow up in size; here the fixed set is the non-negotiable instruction set, so each lifted definition is abstracted into $\{S, K, I\}$ after lifting.)

The structural consequence is more important than the cost. Compilation leaves the supercombinators intact and nested inside one another: in the compiled interpreter of §4, `sp` is a subterm of `step`, `step` of `whnfF`, `rb` of `stepS`, and 40 of the term's 348 distinct nodes are entries of the program's own dictionary. This makes a supercombinator liftable back to its name, and lets a runtime recognize it (§11); a large open term offers neither improvement. Macros are hygienic by default, since a capture is invisible in source and shows up only as a body silently reading the wrong binding; a marked capturing form is reserved for the cases that want it.

5.5 *The standard subject*

A language wants a library, and the library wants to arrive as data rather than as a compiler built-in. We call this a subject in the tradition of subject-oriented programming. The virtual machine applies a loaded program to one argument, the *subject*, which carries the library at fixed axes—the primitives and Y , pairs and projections, Scott lists and numerals, equality on encoded terms, the unquoter, and the interpreters themselves as ordinary supercombinators. (A subject is not the same as an environment: names are resolved in a compile-time table and are gone before anything runs, and no compiled program applies itself to the subject to find its own names.)

5.6 *The quotation line*

So far, SKIjack is a fairly ordinary supercombinator language. The line that separates it from SASL or SKIM is where a term may be turned into data given that quotation is only available at compile time. The form `<τ>` emits the Scott encoding of τ against the enclosing program's object type, computed by the expander; inside the brackets a name is a constructor of that type if it is one and is otherwise inlined and encoded in place. There is no form—*there can be no form*—that takes an arbitrary live value and yields its code. Even if the base representation is always $\{S, K, I\}$, SKIjack and the `whnfF` interpreter are not truly homoiconic, as code can never cross back over into data.

SKIjack refuses explicitly and visibly through its modest type system. Equality is offered on encoded data and refused on functions; a namespace key is a value of a declared path type and not an arbitrary term; a quotation’s argument is source and not an expression to be evaluated first. Each of those would require a function to behave as data, so the checker rejects a program that puts one there before the expander is reached (§9).

6 Execution model

A declaration runs at one of two levels:

- *Level 0*: the compiled term is handed to a reducer as it stands. This is ordinary evaluation: weak reduction is call-by-name, arguments are never forced early, and no form needs thinking. Nothing blocks and nothing can be observed partway; a term either reaches weak head normal form or it does not.
- *Level 1*: a quotation is handed to an interpreter together with a fuel budget, written $\langle t \rangle @n$, or $I \mid - \langle t \rangle @n$ to name an interpreter other than the default. This compiles to an application: the interpreter, its parameters, the Scott numeral, and the quoted program, as the lifted form in §5 shows. Fuel must be explicit because a budget of k permits k step-attempts, of which the last must be the no-redex check, so a computation of s contractions needs $s + 1$. The budget cannot be inferred by the compiler, since a term’s running time is its running time and convergence is only semi-decidable. Elided fuel, written $@[]$, is therefore a *runtime policy* rather than a compile-time value: the budget doubles from a small start until the program produces something other than a timeout or a configured ceiling is reached, and the ceiling is reported as a timeout rather than hidden. (The one case a compiler could fill in is a term it can prove affine, and that case is not interesting enough to special-case.)

The levels differ in what is observable. At level 1, a program’s steps are counted, its divergence is bounded, its errors are values, and its scry operation can block. All of these are available because the interpreter is itself an ordinary term the program may name.

This is a tower in the sense of Smith (1984), with one structural difference that is the subject of this paper. In a reflective tower the levels are crossed at run time: a procedure reifies the environment and continuation it is running under, acts on them as data, and reflects the result back. Over *SKI* the upward crossing is unavailable, since Proposition 3.1 denies any closed term that reifies every live term, so the tower here is declared rather than climbed. There are exactly two levels, the boundary between them is fixed before the program runs, and what a reflective language settles as a design decision this calculus settles as a theorem. Such virtualization is not cheap, in any case: Davis (2027) measured the base interpreter at 340 host contractions for a single object step.

7 Interpreters as declarations

The central move of Davis (2027) was to author the semantics of a new symbol at one site of an interpreter. We performed that move by hand, in a definitional layer with no surface language above it. With SKIjack, this operation becomes simply a declaration. Before extending the interpreter, though, we reproduce it: Listing 2 defines `whnfF` in SKIjack and it compiles atom for atom to the term printed in Appendix B of Davis (2027).

A type declaration names an object language, and from it the compiler generates what an interpreter over that language needs: the constructors and the case form, the spine walker that

```

511 term == S | K | I | App term term
512 maybe == Nothing | Just term
513 whnfF := {
514   step m = sp m nil stepS stepK stepI
515   loop1 f m n2 = step m (Just m) (f n2)
516   loop n m = n Nothing (loop1 loop m)
517 }
518 resS acc c0 c1 c2 = c0 acc
519 resK acc c0 c1 c2 = c1 acc
520 resI acc c0 c1 c2 = c2 acc
521 spApp f acc t u = f t (cons u acc)
522 sp m acc = m (resS acc) (resK acc) (resI acc) (spApp sp acc)
523 rb1 f h x xs = f (App h x) xs
524 rb h args = args h (rb1 rb h)
525 stepI1 x rest = Just (rb x rest)
526 stepI args = args Nothing stepI1
527 stepK2 x y rest = Just (rb x rest)
528 stepK1 x r = r Nothing (stepK2 x)
529 stepK args = args Nothing stepK1
530 stepS3 x y z rest = Just (rb (App (App x z) (App y z)) rest)
531 stepS2 x y r2 = r2 Nothing (stepS3 x y)
532 stepS1 x r = r Nothing (stepS2 x)
533 stepS args = args Nothing stepS1
534

```

Listing 2. The `whnfF` function defined in SKIjack. Pleasingly, this program produces the identical *SKI* artifact as Appendix B of [Davis \(2027\)](#) despite SKIjack not existing at the time of composition.

decomposes an application into a head and an argument list, the rebuilder that puts one back together, a default step equation for each leaf of the object type, and the fuel loop, whose shape is read off the outcome and result types. The timeout constructor is the last terminal of the result type, which for `maybe` is `Nothing`.

The original paper’s observability pair here reduces to two interpreters over a five-constructor object language, differing at the single equation that handles the new leaf. The whole program is seven lines (Listing 3). The first absorbs the new symbol into an error outcome; the second diverges on it. They compile to 768 and 771 atoms, again matching the artifacts of [Davis \(2027\)](#) atom for atom, and they reproduce that paper’s observability table.

The edit is surgical: the two cores differ in one equation, and the compiled terms differ only by what the equation costs: `stepErr` is 4 atoms in the first core and 7 in the second. The difference propagates outward unchanged—`step` is 684 against 687 and `loop1` 736 against 739—and nothing else in either interpreter moves. All the rest, some seven hundred atoms of spine walker, rebuilder, four default step equations and fuel loop, is generated from the three type declarations above them, and generated identically for both. For *SKI* constructions of this shape the compiler is on par with hand construction, at no cost in term size.

```

term5   === S | K | I | App term5 term5 | Err
outcome === Stepped term5 | Done | Errd
result  === RVal term5 | RErr | RTime

omega = S I I (S I I)

wf5Abs := { stepErr acc = Errd }
wf50mg := { stepErr acc = omega }

```

Listing 3. Two interpreters over a five-constructor object language, differing in one equation.

8 The namespace

The artifact of [Davis \(2027\)](#) accepts any term as a “scry path”, or fixed address to value mapping. The language does not inherit that permissiveness, instead restricting valid scry paths to the path type, a list of tagged segments, `path === Nil | Cons seg path`.

The resolver is a parameter of the interpreter core rather than a term closed over inside it. This makes a namespace a namespace: the core is written once, and supplying a different resolver does not recompile it. A namespace literal `ns{}` is sugar for building one. It expands to a chain that compares the incoming path against each key with `EQ5` over the encodings, answering with the oracle’s found-constructor at the first match and its not-yet-constructor when nothing matches, and the chain is an ordinary closed term like any other. [Listing 1](#) and [Table 3](#) show that arrangement as compiled and run.

The namespace handles a negative answer (following [Davis \(2027\)](#)) by providing a not-yet-constructor when no match is found. The interpreter’s outcome and result types each carry an extra constructor that holds the path that blocked, and the driver, meeting one, records the fact and re-runs the program from the top under a resolver that now knows it. While re-running looks wasteful, it guarantees soundness. Because the namespace is append-only, a fact once learned is never withdrawn, so a later run can only get at least as far as an earlier one; the blocked run is not resumed in the middle, where the intermediate state would have to be trusted, but discarded and replayed against more knowledge. We thus require the namespace to be fixed and referentially transparent; if a fact can change, replay proves nothing. Furthermore, fuel is not neutral across a resumption, as a replayed run spends its budget again from the beginning. The fuel a program needs is a function of how many times it blocked, and thus a budget that sufficed on the first attempt may not suffice on the third. (One countenances optimizing this with a more sophisticated data structure.)

8.1 A character table

As a small useful namespace that is more than a toy, we present a character table: the stage of a parser that maps a source character to the term it denotes. It is worth separating because it is the one part of parsing that is a lookup rather than a decomposition, and so the one part that a calculus which cannot take terms apart can still do. It declares the segment type and the facts; we employ the `wfN` interpreter of §7 unchanged. [Listing 4](#) gives the code. The three-fact literal comes to 8,707 atoms. The returns of the four programs are given in [Table 4](#).

In `disc` three lookups are written and two are performed: `K S I` keeps its first argument, so `/src/eye` is discarded by object-level reduction and never read. In `held` three are written

```

613 seg === Src | Ess | Kay | Eye
614
615 chars := ns{/src/ess => <S>, /src/kay => <K>, /src/eye => <I>}
616
617 one    := wfN chars |- <?~/src/ess>@10
618 apply  := wfN chars |- <?~/src/eye ?~/src/ess>@20
619 disc   := wfN chars |- <?~/src/kay ?~/src/ess ?~/src/eye>@40
620 held   := wfN chars |- <?~/src/ess ?~/src/kay ?~/src/eye>@50
621

```

Listing 4. A character table as a namespace, and four programs that read from it.

Table 4. Reading from the character table of Listing 4.

program	atoms	outcome	contractions
one	12,580	RValN S	16,150
apply	15,374	RValN S	62,199
disc	18,203	RValN S	47,257
held	18,283	RValN S (Scry ...) (Scry ...)	16,739

and one is performed: S with two arguments is already in weak head normal form, so the interpreter stops with the two unread lookups still sitting in the returned term as Scry nodes. A scry is thus not an effect that happens when the program mentions it, but a redex like any other, performed only where the reduction order demands a value. The call-by-name discipline of §5 here reaches the namespace as the same phenomenon as the K I Err row of Davis (2027): a symbol present but discarded costs nothing.

A lookup is linear in the position of the fact within the namespace literal, because the literal expands to a chain of EQ5 comparisons tried in order. Reading the first, second and third fact of chars takes 16,150, 30,469 and 45,710 contractions, about fifteen thousand per probe; the small variation between probes is due to the differing size of the segment constructors' own encodings. A namespace literal is therefore an association list with cheap declaration and expensive lookup.

The table supplies resolution, from a name to the term it denotes. Tokenization as the other half requires the source to be a datum on which the program can case rather than a path it can only match; that standard subject (§5) is unbuilt at this stage.

9 Boundary soundness

Proposition 3.1 makes a statement about terms rather than programs. A language owes its users a statement about programs; to wit, a program may not ask the calculus for reification of code as data, and the surface has positions where such a request could be written. Four of them are checked by the data rule:

- (1) An operand of equality.
- (2) The argument of a quotation.
- (3) A key of a namespace literal.
- (4) A scry path.

Equality here is EQ, the glyph name for equality on encoded data. It is an identifier rather than an operator: the checker recognizes the name and guards its operands as the first of the positions above, but the program supplies the term, as Listing 1’s prelude does with EQ5. A declaration may shadow it like any other name. A quotation’s body is checked by the two symbol tables instead, and a case scrutinee and an axis pick are datum positions guarded by their own rules, so the list above is the reach of one check rather than an exhaustive census of the surface.

The check establishes one directional half of what a programmer might want. At each of those positions it refuses every occupant it can *prove* to be a function: a lambda, a bare combinator, a macro, an equation with binders, a core, a namespace literal, an under-applied constructor. It does not establish that an accepted occupant is a datum. With binders untyped, a bound variable’s kind is unknown, so $f\ x = EQ\ x\ \langle K \rangle$ with $g := f\ I$ is accepted although I is a function; the second stage should catch it. We therefore write, “no accepted program contains a provable function in a data position”, rather than “every data position holds a datum”, and the two are separated by the binders.

The check is a pass over the syntax tree that runs before expansion and can only reject, rewriting nothing. So for every program the checker accepts, the emitted term is the same whether the check ran or not, and the laws of §10 are unaffected by the presence of a type discipline. Note that this is erasure of the *checker* and not of the type discipline: declarations are not erased, since they drive generation and fix continuation order.

The implementation enforces and tests this property on every program in its corpus, rather than as a theorem; the formalization of the fragment is the obvious next piece of work. The type discipline itself stops at this stage. A second stage, such as Hindley–Milner over the declared sum types so that $Suc\ K$ is an error rather than a term, is contemplated but unbuilt, and declarations already carry the field types it would need.

10 Implementation and evaluation

An initial compiler is provided as a Python package, *skijack*. Its front end consists of a lexer over a token table and a parser; its middle is the checker of §9 and the generator of §7; its back end is macro expansion to a fixpoint, name resolution, lambda-lifting and bracket abstraction, each a total function on the tree.

Four laws are executable tests. Rendering is canonical, so $expand \circ render = lower$; lifting a term against the dictionary names exact structural matches, so $lower \circ lift = id$; checking can reject but never rewrite, so expansion is the same whether it is on or off; and wherever the suite reads a value out of a compiled term it does so behaviourally, applying the term to fresh markers and observing which one comes back rather than inspecting combinator syntax. SKIjack maintains this lattermost stance as a discipline rather than a property of the dictionary: the dictionary’s own tests are over hashes, tie-breaks and $lower \circ lift$; probing lives where values are read.

Six programs compile by design term-identically to artifacts which Davis (2027) built by hand (Table 5). (A reader may reasonably object that this is circular, since forms generated by the compiler were designed to reproduce those terms. The interpreter can be written out longhand, as twenty ordinary equations, and compiled with generation switched off; it then yields the same 618 atoms. That establishes two things. The generator emits surface syntax and adds no semantics beyond the kernel fold, since the generated and the written source

Table 5. Programs and their atom counts.

program	atoms	
whnfF	618	base interpreter; runs $\langle I \ K \rangle$ in T_0 's 340 contractions
wf5Abs	768	five-constructor interpreter, absorbing equation
wf50mg	771	the same, diverging equation
wfQ	950	blocking interpreter, oracle as a parameter
wfN	1,066	the same over a typed path namespace
EQ5	240	structural equality on encoded terms

compile to one term; and this compiler's bracket abstraction agrees atom for atom, and term for term, with the reference host's bracket abstraction over the same equations. It does not establish independence from the abstraction algorithm shared by both implementations, nor from the equations of [Davis \(2027\)](#) themselves. The longhand source is in fact the generator's own rendered output, kept as user source, isolating the semantics of the comparison rather than its provenance.)

No performance claim is made. The reference host is an oracle and not an implementation with jets: it reduces by substitution, counts every contraction, and is slow on purpose, because its counts are the claims. What a runtime would do with these terms is §11 and, in earnest, an ongoing line of research.

11 The virtual machine

The reference reducer is an oracle, but the language intends to be used and thus wants an implementation. Unusually, what that implementation must be is fixed by a theorem rather than *de gustibus*. The calculus cannot perform two operations for itself: quoting a live term and recognizing one's structure. The first thus belongs to the compiler, which has the syntax tree and quotes at compile time, and the second belongs to the virtual machine, which holds the term on its tape and may factorise it freely ([Jay and Given-Wilson 2011](#)).

The recognition of a fixed set of combinators for native execution follows Turner's extended combinator set and SKIM's microcode ([Clarke et al. 1980](#); [Stoye et al. 1984](#); [Turner 1979b](#)), and a graph reducer with sharing is of the same vintage. The theorem enforces one hard restriction: a jet may never quote on behalf of a running term, because the language guarantees to its users that nothing can.

Two properties of the runtime cut against convenience. Jets change contraction counts, so a jetted run is incommensurable with the counts of [Davis \(2027\)](#). And a reducer that updates a redex in place shares work the reference reducer repeats, which lowers those counts even with no jets installed; the paper's numbers are the reference's substitution semantics, and an implementation that wishes to reproduce them must decline to share. Results must be unaffected either way—and are verified identical in the experiments.

12 Limitations

SKIjack has evaluation of encoded terms, including terms built at run time. However, even with `eval`, it lacks *reification* of a live value, for reasons discussed in §5.1. The line is visible in the compiler, and it does not fall where a reader coming from a homoiconic language may expect. Writing an encoding down or naming it is ordinary. Given the definition $k_2 = K \ K$,

the program `whnfF |- <k2>@10` compiles without complaint, at 741 atoms; so does `whnfF |- <I K>@10`, at 739. In both cases the compiler knows before anything runs which term the quotation contains, and emits its encoding.

However, the same quotation over a parameter does not compile:

```
f x = <x>
```

Stage A refuses it, reporting that `x` “names neither table: it is not a constructor of the object type term and not a level-0 name”. Had `f` been applied to `k2`, the datum the program is asking for is the one that `<k2>` already emits; the compiler can write that encoding, and does, two lines earlier. What differs is *when* the term is known. A level-0 name is resolved before anything runs, so quoting it is a compile-time lookup; a parameter is bound while something runs, and recovering a term from the value it has been given is the operation the calculus simply does not possess.

In this sense, *reify* is the missing operation. Evaluation is available and needs no special support: an interpreter is an ordinary term, and applying it to a datum is an ordinary application. Reification would have to run the other way, from a live value to a datum describing it, and Proposition 3.1 denies any closed term that does so for every argument. SKIjack does not *forbid* reification so much as decline to feign it: quotation is a compile-time construct, the checker of §9 refuses the occupants it can prove to be functions, and what remains is a language in which the staging line is visible in the syntax rather than discovered at run time.

The language is not self-hosting, which capability is not currently a design criterion for SKIjack. This would be a smaller undertaking than it may sound—the expander’s input is a syntax tree, which is Scott-encodable like any other datum, so the expander is separable from the parser and text handling could stay where it is. What is foregone by declining is the Futamura projections (Futamura 1971; Jones et al. 1993) inside the calculus: an expander that is a term could be run under an interpreter and specialized, and the tower would gain a level with new content rather than another copy of the same interpreter. Nothing else here depends on it, and the compiler is metalevel by theorem in any case (§9), so self-hosting would relocate quotation rather than remove it.

The language’s type discipline stops at the stage described in §9. The standard subject of §5 is specified but unbuilt at time of submission, so library names are inlined and level-1 programs do not share a quoted library. Qualified names parse and do not yet resolve.

While the language is Turing-complete, certain functions are transferred to the virtual machine host, such as scaffolding the initial evaluation, preserving the state and memory arena, and providing the supercombinator jet acceleration table.

13 Conclusion

SKIjack is a language whose feature set was read off an existing artifact rather than designed in advance, like the assembly language of old. Three consequences follow from that discipline. The declaration of an object language generates the furniture an interpreter over it requires, so authoring the semantics of a new symbol is an edit to a single equation: the two interpreters of §7 differ in one line and in three atoms. The generation costs nothing over manual construction; six programs compile to the artifacts of Davis (2027) atom for atom. Finally, a namespace is an ordinary core parameter rather than a primitive, which is what makes its cost legible: a literal is an association list, and a hit is linear in where the fact sits.

This is not, of course, performant! The reference host is an oracle; what a runtime would do with these terms is §11 and, in earnest, other work. The standard subject of §5.5 is specified and unbuilt, and with it the tokenization half of §8.1. Self-hosting is declined deliberately at the current time, per §12. Rather, the exploration of supercombinator techniques by SKIjack and its virtual machine(s) may spur further reflection and development in the field of efficient functional language implementation, as well as reconceptualization of the role of combinatory logic in the foundations of computer science and verifiable computing.

Data availability statement

A repository containing the specification and an implementation of the language used to produce the results in this paper is available at <https://github.com/sigilante/skijack>. This requires only the Python package `aviary-kernel`.

References

- Alessandro Berarducci and Corrado Böhm. 1993. A Self-Interpreter of Lambda Calculus Having a Normal Form. In *Computer Science Logic (CSL '92) (Lecture Notes in Computer Science, Vol. 702)*. Springer, 85–99.
- Corrado Böhm. 1954. Calculatrices digitales: du déchiffre de formules logico-mathématiques par la machine même dans la conception du programme. *Annali di Matematica Pura ed Applicata* 37 (1954), 175–217. Series 4. Originally the author's doctoral dissertation, ETH Zürich, 1951.
- Corrado Böhm. 1968. *Alcune proprietà delle forme β - η -normali nel λ -K-calcolo*. Technical Report 696. Istituto per le Applicazioni del Calcolo, Rome.
- Corrado Böhm and Alessandro Berarducci. 1985. Automatic synthesis of typed Λ -programs on term algebras. *Theoretical Computer Science* 39 (1985), 135–154.
- Corrado Böhm and Wolf Gross. 1966. Introduction to the CUCH. In *Automata Theory*, Eduardo R. Caianiello (Ed.). Academic Press, New York, 35–65.
- T. J. W. Clarke, P. J. S. Gladstone, C. D. MacLean, and A. C. Norman. 1980. SKIM — The S, K, I Reduction Machine. In *Proceedings of the 1980 ACM Conference on LISP and Functional Programming*. ACM.
- N. E. Davis. 2027. A self-interpreter for *SKI*: Authoring semantics for new symbols. *Journal of Functional Programming* (2027). In review.
- Yoshihiko Futamura. 1971. Partial Evaluation of Computation Process: An Approach to a Compiler-Compiler. *Systems, Computers, Controls* 2, 5 (1971), 45–50. Reprinted in *Higher-Order and Symbolic Computation* 12(4):381–391, 1999.
- R. John M. Hughes. 1982. Super-Combinators: A New Implementation Method for Applicative Languages. In *Proceedings of the 1982 ACM Symposium on LISP and Functional Programming*. ACM.
- Jan Martin Jansen. 2013. Programming in the λ -Calculus: From Church to Scott and Back. In *The Beauty of Functional Code*. Lecture Notes in Computer Science, Vol. 8106. Springer, 168–180.
- Barry Jay and Thomas Given-Wilson. 2011. A Combinatory Account of Internal Structure. *Journal of Symbolic Logic* 76, 3 (2011), 807–826.
- Thomas Johnsson. 1984. Efficient Compilation of Lazy Evaluation. In *Proceedings of the SIGPLAN Symposium on Compiler Construction*. ACM.
- Neil D. Jones, Carsten K. Gomard, and Peter Sestoft. 1993. *Partial Evaluation and Automatic Program Generation*. Prentice Hall International.
- Mike S. Joy, V. J. Rayward-Smith, and F. Warren Burton. 1985. Efficient Combinator Code. *Computer Languages* 10, 3–4 (1985).
- Donald E. Knuth and Luis Trabb Pardo. 1980. The Early Development of Programming Languages. In *A History of Computing in the Twentieth Century*, N. Metropolis, J. Howlett, and Gian-Carlo Rota (Eds.). Academic Press, New York, 197–273.
- John McCarthy. 1960. Recursive Functions of Symbolic Expressions and Their Computation by Machine, Part I. *Commun. ACM* 3, 4 (1960), 184–195.
- Kohei Noshita. 1985. Translation of Turner Combinators in $O(n \log n)$ Space. *Inform. Process. Lett.* 20, 2 (1985).
- Simon L. Peyton Jones. 1987. *The Implementation of Functional Programming Languages*. Prentice Hall.

```

868 decl      := type-decl | sig | equation | macro | core | def | run
869 run       := NAME ':' [expr '|-' ] '<' expr '>' ('@' NUMBER | '@[]')
870 type-decl := NAME '===' ctor ('|' ctor)*
871 ctor      := CNAME TYPE*                -- CNAME capitalized; fields are types
872 sig       := NAME ':' TYPE ('->' TYPE)*   -- optional; ignored by the expander
873 equation  := NAME NAME* '=' expr         -- inside a core, or at top level
874 core      := NAME NAME* ':' '{' equation* '}' -- parameters after the name
875 macro     := NAME NAME* ':=*' expr | NAME NAME* '!=' expr
876 def       := NAME ':' expr
877 expr      := app
878 app       := atom+                        -- left-associative
879 atom      := NAME | GLYPH | '(' expr ')'
880           | '[' expr expr+ ']'           -- cell
881           | '<' expr '>'                  -- quotation: a datum
882           | '?^' atom                    -- scry
883           | NUMBER '@' atom              -- axis pick
884           | '\' NAME '.' expr
885           | expr '|>' '{' branch ';' branch)* '}'
886           | 'ns{' fact (',' fact)* '}'
887 branch    := CNAME NAME* expr
888 fact      := path '=>' expr
889 path      := ('/' NAME)+

```

Listing 5. The grammar of the SKIjack language.

- Dana S. Scott. 1962. A System of Functional Abstraction. (1962). Lectures delivered at the University of California, Berkeley, 1962/63; unpublished manuscript, Stanford University, 1963.
- Brian Cantwell Smith. 1984. Reflection and Semantics in Lisp. In *Proceedings of the 11th ACM SIGACT-SIGPLAN Symposium on Principles of Programming Languages (POPL '84)*. 23–35.
- William R. Stoye. 1985. *The Implementation of Functional Languages Using Custom Hardware*. Ph. D. Dissertation. University of Cambridge Computer Laboratory. Technical Report 81.
- William R. Stoye, T. J. W. Clarke, and A. C. Norman. 1984. Some Practical Methods for Rapid Combinator Reduction. In *Proceedings of the 1984 ACM Symposium on LISP and Functional Programming*. ACM.
- David A. Turner. 1979a. Another Algorithm for Bracket Abstraction. *The Journal of Symbolic Logic* 44, 2 (1979), 267–270.
- David A. Turner. 1979b. A New Implementation Technique for Applicative Languages. *Software: Practice and Experience* 9, 1 (1979), 31–49.
- Curtis Yarvin. 2018. Nock 4K. Tlon Corporation, <https://docs.urbit.org/language/nock/reference/definition>. Accessed 2024-02-20.

A Language description

SKIjack’s grammar is given in Listing 5; newlines terminate a declaration and an equation, and are invisible inside brackets, a case block and a namespace literal.

919
920
921
922
923
924
925
926
927
928
929
930
931
932
933
934
935
936
937
938
939
940
941
942
943
944
945
946
947
948
949
950
951
952
953
954
955
956
957
958
959
960
961
962
963
964
965
966
967
968
969

whnfF compiles to 618 atoms, identical to Appendix B of [Davis \(2027\)](#) (Listing 6).

Listing 6. The `whnfF` function fully expanded.

Listing 6. The `whnfF` function fully expanded.